



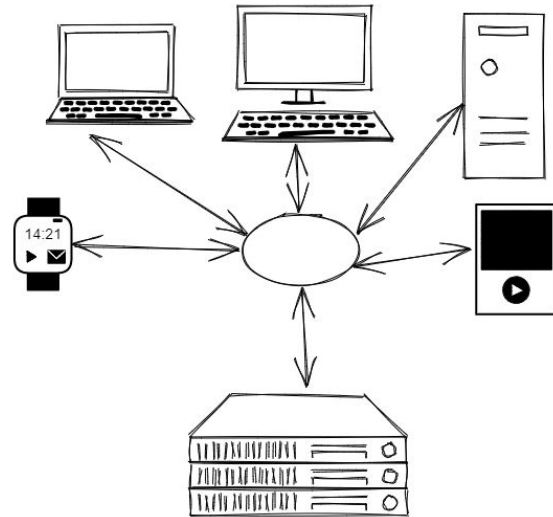
APIs alternativas 1

Comunicación Cliente Servidor

Como ya hablamos la comunicación básica en un sistema se da entre dos nodos generando una comunicación “cliente servidor”.

Estos roles tradicionalmente tomados por “computadoras” ahora son tomados por diversos dispositivos:

- » Dispositivos móviles.
- » Sensores.
- » IoT.
- » Electrodomésticos.
- » Maquinarias.



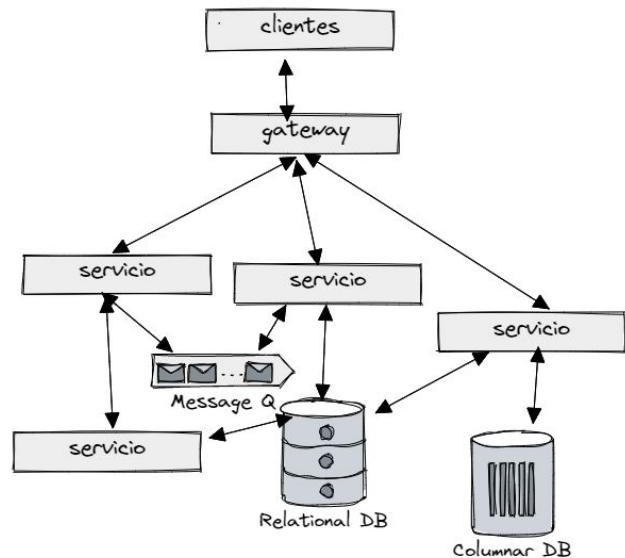
Comunicación Cliente Servidor

Y los “servicios” se van complejizando de manera que los mismos se resuelven a través de la colaboración de diversos componentes, que se comunican entre ellos.

Esto provoca que existan diversas formas de establecer comunicación entre dos puntos de un sistema.

gRPC nos sirvió como ejemplo para analizar internamente una manera de comunicación.

Hoy veremos algunos de los estándares alternativos más populares para generar APIs de consumo de servicios.





APIs

La forma de la API es el punto central a definir ya que es la que indica la semántica y las operaciones posibles a la hora de consumir un servicio.

Para implementarlas existen diversos paradigmas, estándares, frameworks, y tecnologías a elegir.





APIs

A pesar de que cualquiera de ellas puede utilizarse para resolver casi cualquier API se pueden tomar diversos criterios para elegir la “mejor” para el servicio que estamos implementando.

- Formas de comunicación y periodicidad.
 - Transporte.
 - Diversidad de clientes, lenguajes, tecnologías.
 - Maneras de describir las operaciones.
 - Diversidad de los payloads.
- 

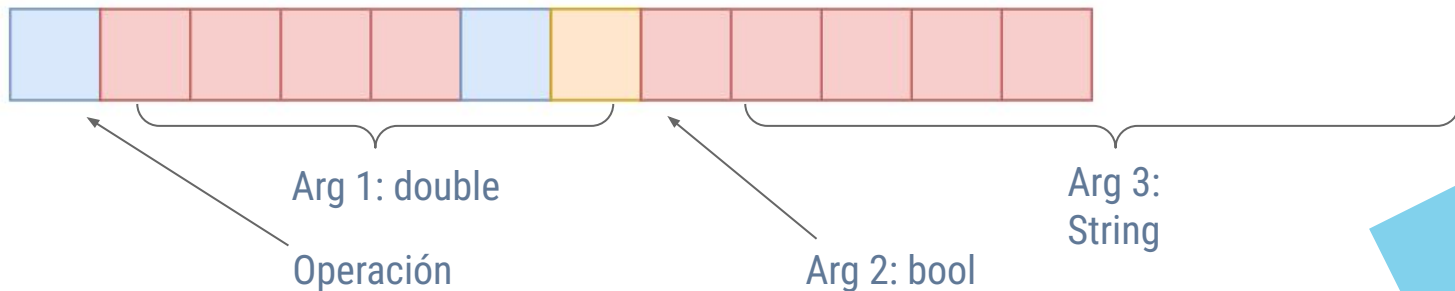


Tira de Bytes

Tira de Bytes

La tira de Bytes es la forma más “primitiva” y/o “básica” de generar una API.

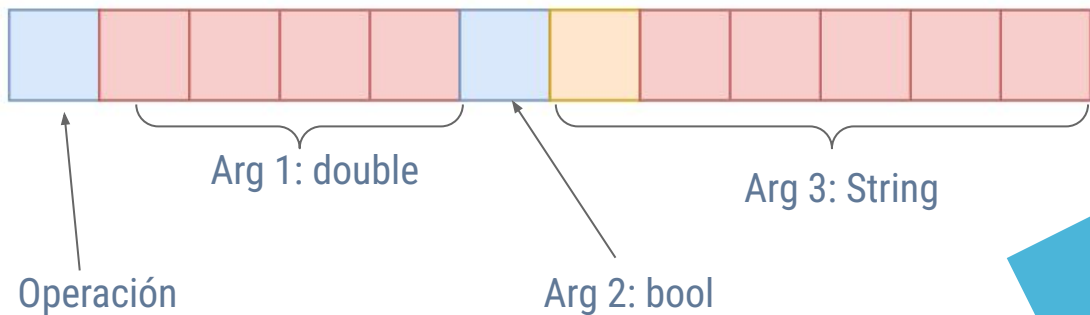
Durante mucho tiempo fue la única opción hasta que se generaron nuevas y mejores alternativas.



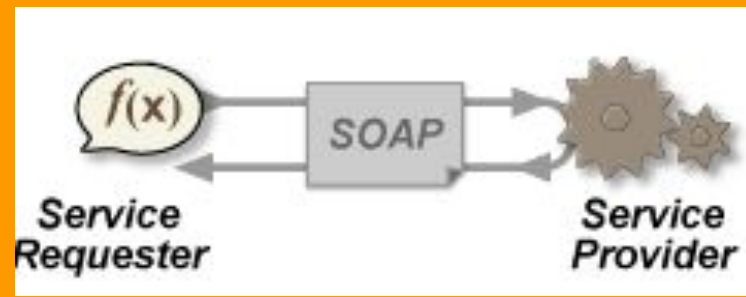
Tira de Bytes

Pero al día de hoy todavía pueden encontrarse algunas API legacy que usan alguna variante de la misma.

Se trata de abrir una conexión por socket y convenir (mediante el uso de documentación) de que manera se envía los diferentes valores del request y la response.



SOAP





SOAP

SOAP (Simple Object Access Protocol) es un protocolo estándar de comunicación mediante el intercambio de datos utilizando XML.



SOAP

Ejemplo de un mensaje en xml.

```
<?xml version="1.0"?>
<soap:Envelope
  xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
  soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">

  <soap:Body xmlns:m="http://www.example.org/stock">
    <m:GetStockPrice>
      <m:StockName>IBM</m:StockName>
    </m:GetStockPrice>
  </soap:Body>

</soap:Envelope>
```



SOAP - Características

La definición de los servicios también se hace mediante XML.

El protocolo tiene 3 características principales:

- **Extensibilidad:** El protocolo pueden extenderse para hacer servicios complejos o agregar seguridad y routing
 - **Neutralidad:** Utiliza transporte por TCP y puede aplicarse diversos protocolos de aplicación como HTTP, SMTP o JMS.
 - **Independencia:** (permite cualquier modelo de programación) al definirse vía XML se puede programar en diversos lenguajes y paradigmas.
- 



SOAP - Mensaje

Un mensaje en SOAP consta de las siguientes partes:

- **Envelope:** Elemento que engloba
 - **Header:** Elemento opcional que tiene atributos que son complementarios al mensaje, por ejemplo, elementos de autenticación.
 - **Body:** Elemento obligatorio con el contenido del mensaje a enviar
 - **Fault:** Información de errores que se produjeron (si se produjeron).
- 

SOAP - Mensaje

Ejemplo de la estructura de un mensaje.

```
<?xml version="1.0"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV =
"http://www.w3.org/2001/12/soap-envelope"
    SOAP-ENV:encodingStyle =
"http://www.w3.org/2001/12/soap-encoding">

    <SOAP-ENV:Header>
        ...
    </SOAP-ENV:Header>
    <SOAP-ENV:Body>
        ...
        <SOAP-ENV:Fault>
            ...
        </SOAP-ENV:Fault>
        ...
    </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

SOAP - Definición de un servicio

La definición de los mensajes, puertos y operaciones.

```
<wsdl:message name="getMessageRequest">
  <wsdl:part element="impl:getMessage" name="parameters">
  </wsdl:part>
</wsdl:message>

<wsdl:message name="getMessageResponse">
  <wsdl:part element="impl:getMessageResponse" name="parameters"></wsdl:part>
</wsdl:message>

<wsdl:portType name="Greetings">
  <wsdl:operation name="getMessage">
    <wsdl:input message="impl:getMessageRequest" name="getMessageRequest">
    </wsdl:input>
    <wsdl:output message="impl:getMessageResponse"
name="getMessageResponse">
    </wsdl:output>
  </wsdl:operation>
</wsdl:portType>
```

SOAP - Definición de un servicio

WSDL es el formato en XML con el cual se define el servicio

```
<wsdl:types>
  <schema elementFormDefault="qualified" targetNamespace="http://DefaultNamespace"
    xmlns="http://www.w3.org/2001/XMLSchema">
    <element name="getMessage">
      <complexType>
        <sequence>
          <element name="message" type="xsd:string"/>
        </sequence>
      </complexType>
    </element>
    <element name="getMessageResponse">
      <complexType>
        <sequence>
          <element name="getMessageReturn"
            type="xsd:string"/>
        </sequence>
      </complexType>
    </element>
  </schema>
</wsdl:types>
```


SOAP - Definición de un servicio

Y la definición de los bindings y el servicio.

```
<wsdl:binding name="GreetingsSoapBinding" type="impl:Greetings">
  <wsdlsoap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
  <wsdl:operation name="getMessage">
    <wsdlsoap:operation soapAction=""/>
    <wsdl:input name="getMessageRequest">
      <wsdlsoap:body use="literal"/>
    </wsdl:input>
    <wsdl:output name="getMessageResponse">
      <wsdlsoap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
</wsdl:binding>

<wsdl:service name="GreetingsService">
  <wsdl:port binding="impl:GreetingsSoapBinding" name="Greetings">
    <wsdlsoap:address location="http://localhost:8080/WebServiceSoap/services/Greetings"/>
  </wsdl:port>
</wsdl:service>
```



SOAP - Generar código

Los lenguajes proveen de compiladores que permiten interpretar las definiciones del xml y generar las clases de servicios y stubs para que luego se pueda completar la implementación del servicio.

El proceso es similar al visto con gRPC, varían las clases a implementar y los nombres.

En Java existen librerías como CXF (más popular), AXIs y soporte nativo vía JAX-WS. La necesidad de usar uno u otro puede estar dada por el código existente.





SOAP - Ventajas e inconvenientes

Durante mucho tiempo SOAP fue el sistema de web services más popular debido a sus características de extensibilidad y multi lenguaje.

Las contra que tiene son que la definición via xml, puede ser muy “verbosa”, que la definición de servicios puede ser compleja (aún para servicios simples).





SOAP - Ventajas e inconvenientes

Entonces fue perdiendo popularidad contra REST y se generó una dicotomía donde
REST se usa para servicios “simples” web y B2C;
y SOAP se usa para aquellos más complejos con necesidades de mayor seguridad como Financieras y B2B





Rest



Rest

Según la definición original escrita en la tesis doctoral de Roy T. Fielding REST (representational state transfer) es:

“Estilo arquitectónico para sistemas distribuidos de hipermedia, describiendo los principios de ingeniería de software que guían REST y las constantes de interacción elegidas para retener esos principios”

Entonces originalmente era un conjunto de principios de arquitectura. Como Fielding trabajó en el protocolo HTTP y los principios de REST están muy atados a ese protocolo el REST tomó un significado más amplio.





Rest

Entonces en estos tiempos se podría describir a Rest no oficialmente como

Una interfaz remota de servicios basada en **Recursos** sobre el protocolo HTTP. Esta interfaz se utiliza para obtener y modificar datos multimedia y texto en diversos formatos como JSON, XML, binario, etc.

Rest probablemente sea la interfaz web más popular por esa conexión con HTTP.

La principal diferencia con otros tipos de interfaces como RPC es la definición de los servicios a través de **Recursos** (sustantivos) en vez de **Verbos** (acciones).





Rest - Principios de Arquitectura

1. **Arquitectura cliente-servidor (Client-Server):** Para lograr separación de responsabilidades y desacoplar la interacción entre el cliente y el servidor. Como la dependencia es a través del mensaje modificar cada componente no debería afectar al otro.
- 



Rest - Principios de Arquitectura

2. **Ausencia de estado (Stateless):** El estado/contexto de la petición es mantenida por el cliente. El servidor es stateless y recibe todo del cliente. Esto va a permitir que diversas instancias del servicio puedan atender llamadas sucesivas de un mismo cliente.
- 



Rest - Principios de Arquitectura

3. **Habilitación y uso del cache (Cacheable):** Todos los recursos deben ser cacheables, mientras y por el tiempo que tenga sentido. Esto permite optimizar la comunicación entre el cliente y el servidor
- 



Rest - Principios de Arquitectura

4. **Sistema por capas (Layered):** Un cliente debe conocer únicamente la capa a la que le está hablando. Es decir, no debe tener en cuenta aspectos concretos





Rest - Principios de Arquitectura

5. **Ejecución de código (Code on Demand):** Esto era opcional y con el tiempo entró en desuso. Era la posibilidad de poder ejecutar código en el cliente (o desde el cliente) mediante el uso de scripts o applets
- 



Rest - Principios de Arquitectura

6. **Interfaz uniforme:** se aplica el principio de “generalidad” para simplificar las interacciones. Para ello define las siguientes restricciones:
 - a. **Identificación de recursos en las peticiones:** Todos los recursos se identifican unívocamente. Esto es independiente de cual es la representación que se envía (que se puede solicitar) por medio de headers.
 - b. **Manipulación de recursos a través de representaciones:** Cuando el cliente obtiene la información de un recurso la misma debe contener toda la información para poder modificar/borrar a dicho recurso.
 - c. **Mensajes auto-descriptivos:** cada mensaje debe contener toda la información necesaria para que se ejecute.
 - d. **Hipermedia como motor del estado de la aplicación:** a partir de un pedido inicial el cliente debería poder “descubrir” todos los servicios del
- 



Rest - Recursos

Los recursos son la información que diferentes aplicaciones proporcionan a sus clientes. Los recursos pueden ser imágenes, videos, texto, números o cualquier tipo de datos.





Rest - Recursos

Los recursos pueden ser de 3 tipos:

- **colección:** un conjunto de elementos que tienen características similares. Se identifican con el sustantivo que la describe (generalmente plural).
 - **singleton/documento:** una instancia individual de una colección. Se identifica mediante un identificador natural o interno y contiene el resto de los datos que contienen.
 - **subcolecciones:** son conjuntos de singletons que están subordinados a un elemento.
- 



Rest - Recursos - URIs

Para acceder a los recursos se utilizan URIs.

Partiendo de la uri del servicio y potencialmente un contexto y versionado, por ejemplo: <http://remote-service/context/v1>

Jerárquicamente se agrega componentes

- **colecciones:** con el nombre del recurso: `/movies/`
 - **singleton:** con la colección y el identificador: `/movies/{id}`
 - **subcolección:** agregamos la colección: `/movies/{id}/actors`
- 



Rest - Recursos - URIs

Se utilizan las features de HTTP para hacer más específicos el pedido:

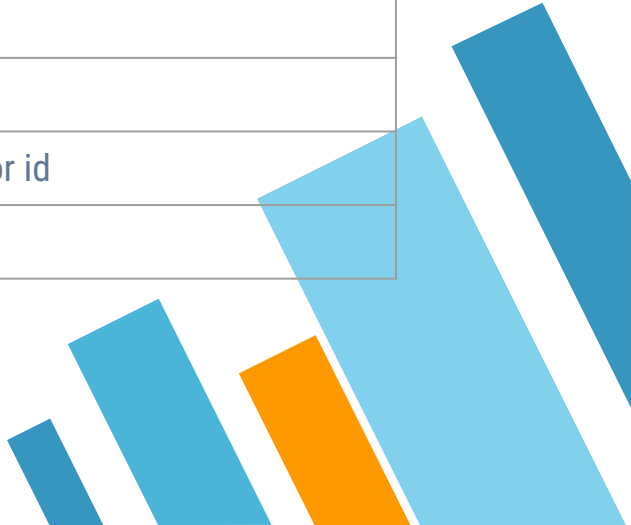
- **status codes:** para indicar resultados.
 - **content-types:** para pedir/indicar la representación a usar en el recurso
 - **query params:** para agregar limitantes como criterios de búsqueda, paginación, etc.
- 



Rest - Recursos - Métodos

Los métodos HTTP permiten realizar acciones sobre los recursos como búsquedas y las operaciones CRUD.

GET	/movies	Trae todas las películas
POST	/movies	Agrega una película
GET	/movies/{id}	Trae la película identificada por id
PUT	/movies/{id}	Modifica la película identificada por id
DELETE	/movies/{id}	Borra la película identificada por id



Rest - Recursos - HATEOAS

Hipermedia como motor del estado de la aplicación es una de las restricciones de REST.

Básicamente indica que cada cliente debería poder navegar el servicio mediante la respuesta al primer Request. La manera de navegar es agregar a la respuesta hipermedia links a los recursos dependientes (subcolecciones o singletons).

```
{
  "movieID": 101001,
  "movieName": "Nueve Reinas",
  "year": 2000,
  "directorId": 4040,
  "links": [
    { "href": "101001/reviews", "rel": "reviews", "type" : "GET" },
    { "href": "directors/4040", "rel": "director", "type" : "GET" },
    { "href": "101001", "rel": "delete", "type" : "DELETE" }
  ]
}
```



Rest - Código

La mayoría de los lenguajes tienen frameworks que funcionan como middleware para poder crear aplicaciones REST simples.

Dentro del mundo Java tenemos varios, los ejemplos presentados a continuación combinan elementos de Spring - Jersey - JAX-RS para poder proveer un servicio que retorna un recurso llamado message representado en JSON



Rest - Código - Objeto

Definición del objeto que representa al Recurso, en este caso un mensaje

```
public record Message(String message, LocalDateTime date) {  
  
    public Message(String message) {  
        this(message, LocalDateTime.now());  
    }  
}
```

Rest - Código - Controlador

Definición del componente que responde los pedidos HTTP

```
@Component
@Path("/messages")
public class MessageResource {

    @Autowired
    private MessageService messageService;

    @POST
    @Produces("application/json")
    public Message message(@QueryParam("message") String message) {
        return messageService.create(message);
    }
}
```



Rest - Código - Configuración

Configuración para registrar el controlador en el framework Jersey

```
@Component
public class JerseyConfig extends
ResourceConfig {
    public JerseyConfig() {
        register(MessageResource.class);
    }
}
```



Rest - Código

Se define una aplicación de Spring Boot con un bean servicio del tipo **MessageService**

```
@SpringBootApplication
public class RestTestApplication {

    @Bean
    MessageService buildMessageService() {
        return new DefaultMessageService();
    }

    public static void main(String[] args) {
        SpringApplication.run(RestTestApplication.class, args);
    }
}
```




Rest - Ventajas

REST al ser uno de los estándares más populares tiene un gran soporte en lenguajes, frameworks y librerías. Además de conocimiento colectivo entre la comunidad de desarrollo:

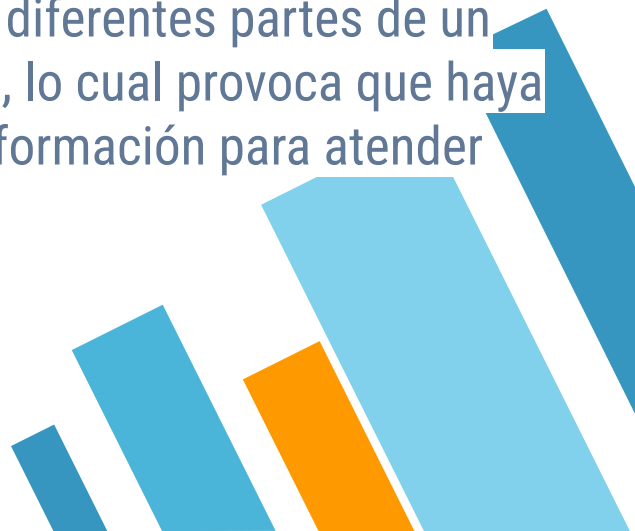
Por las restricciones que lo definen está armado para que sea:

- **escalable:** stateless, client-server, cacheable
 - **flexible:** client-server y las formas de las representaciones
 - **independiente de la tecnología:** como el transporte es http tanto cliente y servidor pueden estar escrito en lenguajes diferentes
- 



Rest - Desventajas

En contrapartida algunas desventajas de REST puede ser

- Al no haber un estándar definido surgen inconvenientes cuando se quieren describir sistemas o partes que no son fácilmente describibles a través de sustantivos (y/o operaciones CRUD).
 - En ciertas situaciones se vuelve incómodo cuando diferentes partes de un sistema requieren diferentes facetas de un recurso, lo cual provoca que haya varios endpoints o uno que devuelva demasiada información para atender todos los casos.
- 



CREDITS

Content of the slides:

- » POD - ITBA

Images:

- » POD - ITBA
- » Or obtained from: commons.wikimedia.org

Slides theme credit:

Special thanks to all the people who made and released these awesome resources for free:

- » Presentation template by [SlidesCarnival](https://slidescarnival.com)
 - » Photographs by Unsplash
- 